

DL Assignment 3 — Handwritten/Alphabet Recognition using MLP Classifier

This assignment performs:

Alphabet Letter Recognition using Neural Network (MLP)

using:

- Scikit-learn
- MLPClassifier
- Letter Recognition dataset

1. What We Have To Do?

We have to:

1. load letter recognition dataset
2. preprocess dataset
3. scale features
4. build MLP neural network
5. train model
6. predict alphabet letters
7. evaluate accuracy
8. display confusion matrix

Main Goal

To classify:

English alphabet letters (A–Z)

based on numerical image features.

Example

Input features represent:

shape properties of handwritten letters

Output:

A, B, C, D ... Z

2. What Type of Problem is This?

This is:

Multi-Class Classification Problem

Because output contains:

26 classes

Class	Meaning
-------	---------

A	Letter A
---	----------

B	Letter B
---	----------

...	...
-----	-----

3. What is MLP?

MLP =

Multi-Layer Perceptron

A type of:

Artificial Neural Network (ANN)

Structure of MLP

Input Layer



Hidden Layer(s)



Output Layer

In This Assignment

16 input features



Hidden Layer (100 neurons)



Hidden Layer (100 neurons)



26 output classes

4. Overall Workflow

Load Dataset



Split Dataset



Feature Scaling



Build MLP Model



Train Model



Predict Letters



Evaluate Accuracy



Display Confusion Matrix

CELL 1 — Install Libraries

!pip install pandas scikit-learn yellowbrick

Purpose

Installs required libraries.

Libraries Used

Library	Purpose
pandas	dataset handling
scikit-learn	machine learning
yellowbrick	visualization
matplotlib	graphs

CELL 2 — Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

Line-by-Line Explanation

pandas

import pandas as pd
Used for:

handling dataset

train_test_split

from sklearn.model_selection import train_test_split
Splits data into:

- training set
- testing/validation set

StandardScaler

from sklearn.preprocessing import StandardScaler
Used for:

feature scaling

MLPClassifier

from sklearn.neural_network import MLPClassifier
Creates:

neural network classifier

accuracy_score

from sklearn.metrics import accuracy_score
Measures:

prediction accuracy

CELL 3 — Download Dataset

!wget <https://archive.ics.uci.edu/ml/machine-learning-databases/letter-recognition/letter-recognition.data>

Purpose

Downloads:

Letter Recognition Dataset

from UCI repository.

Dataset Contains

Data	Meaning
Features	letter image properties
Label	alphabet letter

CELL 4 — Load Dataset

```
dataset = pd.read_csv("letter-recognition.data", header=None)
```

Purpose

Loads dataset into:

pandas dataframe

header=None

Dataset has:

no column names

CELL 5 — Separate Features and Labels

```
X = dataset.iloc[:, 1:17]
```

```
Y = dataset.iloc[:, 0].values.ravel()
```

Explanation

X

```
dataset.iloc[:, 1:17]
```

Selects:

feature columns

Why 1:17 ?

Because:

- column 0 = target letter
- columns 1–16 = input features

Y

```
dataset.iloc[:, 0]
```

Stores:

output labels

like:

A, B, C ...

.ravel()

Converts:

2D → 1D array

Required for classifier.

CELL 6 — Train-Test Split

```
X_train, X_validation, Y_train, Y_validation = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)
```

Purpose

Splits dataset into:

- training data
- validation/testing data

test_size=0.2

Means:

Dataset	Percentage
---------	------------

Training	80%
----------	-----

Validation	20%
------------	-----

random_state=42

Ensures:

same split every run

for reproducibility.

CELL 7 — Feature Scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_validation = scaler.transform(X_validation)
```

Why Scaling Needed?

Different features have different ranges.

Scaling improves:

- convergence
- training speed
- neural network stability

Standardization Formula

$$z = \frac{x - \mu}{\sigma}$$

fit_transform()

Does:

1. calculate mean/std
2. scale training data

transform()

Uses training statistics on validation data.

Validation data should NOT influence training.

CELL 8 — Build MLP Model

```
mlp = MLPClassifier(  
    hidden_layer_sizes=(100, 100),  
    max_iter=300,  
    activation='relu',  
    solver='adam',  
    random_state=42  
)
```

MOST IMPORTANT CELL

Creates:

neural network model

hidden_layer_sizes=(100,100)

Means:

2 hidden layers

Each hidden layer contains:

100 neurons

Architecture

Input Layer



Hidden Layer 1 (100)



Hidden Layer 2 (100)



Output Layer (26 classes)

max_iter=300

Maximum:

300 training iterations

activation='relu'

Uses ReLU activation.

Formula:

$f(x) = \max(0, x)$

Why ReLU Used?

Advantages:

- fast
- avoids vanishing gradient
- efficient learning

solver='adam'

Adam optimizer updates weights efficiently.

random_state=42

Ensures reproducible results.

CELL 9 — Train Model

```
mlp.fit(X_train, Y_train)
```

Purpose

Trains neural network.

During Training

Model:

1. predicts letters
2. calculates error
3. updates weights
4. improves predictions

CELL 10 — Import Confusion Matrix Visualizer

```
from yellowbrick.classifier import ConfusionMatrix
```

Purpose

Used to:

visualize confusion matrix

CELL 11 — Define Classes

```
classes = list("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
```

Purpose

Stores:

all alphabet labels

CELL 12 — Create Confusion Matrix Object

```
cm = ConfusionMatrix(mlp, classes=classes)
```

Purpose

Creates:

graphical confusion matrix visualizer

CELL 13 — Fit and Score Confusion Matrix

```
cm.fit(X_train, Y_train)
cm.score(X_validation, Y_validation)
cm.show()
```

cm.fit()

Trains visualizer.

cm.score()

Predicts validation data and computes matrix.

cm.show()

Displays confusion matrix graph.

Confusion Matrix Meaning

Rows:

actual letters

Columns:

predicted letters

Good Confusion Matrix

Strong diagonal values mean:

correct predictions

The confusion matrix visualization is shown on page 2 of the uploaded PDF.

CELL 14 — Predictions

```
predictions = mlp.predict(X_validation)
print(predictions)
```

Purpose

Predicts:

alphabet letters

for validation data.

Example Output

```
['X' 'L' 'A' ...]
```

CELL 15 — Accuracy

```
print("Accuracy:", accuracy_score(Y_validation, predictions))
```

Accuracy Formula

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Output

Accuracy: 0.958

Meaning:

95.8% predictions correct

Important Concepts Used

Concept	Purpose
ANN/MLP	classification
Feature Scaling	normalize input
ReLU	activation function
Adam Optimizer	weight updates
Confusion Matrix	performance analysis

Important Viva Questions

Q1. What is MLP?

Multi-Layer Perceptron neural network.

Q2. What type of problem is this?

Multi-class classification.

Q3. Why feature scaling used?

To normalize feature ranges and improve learning.

Q4. What does `hidden_layer_sizes=(100,100)` mean?

Two hidden layers with 100 neurons each.

Q5. Why ReLU used?

Fast and efficient activation function.

Q6. What does Adam optimizer do?

Updates weights efficiently during training.

Q7. What is confusion matrix?

Table showing actual vs predicted classes.

Q8. What does accuracy 0.958 mean?

95.8% predictions are correct.

Short Practical Explanation

This program performs alphabet letter recognition using an MLP neural network classifier. The dataset is preprocessed using feature scaling, and a neural network with two hidden layers is trained using the Adam optimizer and ReLU activation function. The model predicts alphabet classes and is evaluated using accuracy and confusion matrix visualization.